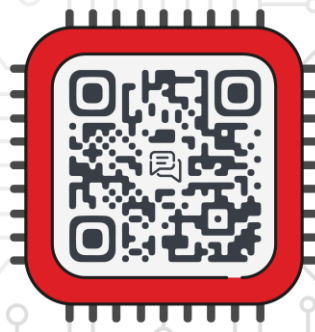




3



20



2400 sp

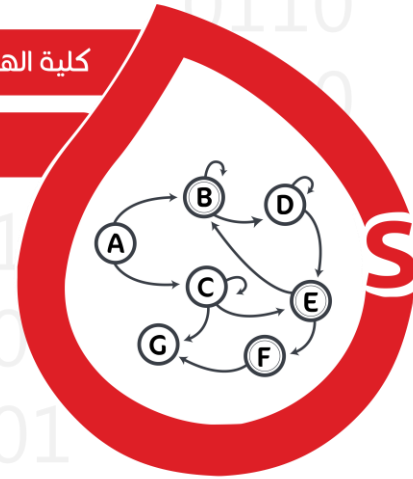
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

NFAs & ϵ NFAs

د. هبة حاتم



اللغات الصورية

RB Informatics; 01/05/2025

محتوى مجاني غير مخصص للبيع التجاري

Extension of δ to NFA paths

- Basic: $\hat{\delta}(q, \epsilon) = \{q\}$
- Induction:
 - Let $\hat{\delta}(q_0, w) = \{p_1, p_2, \dots, p_k\}$
 - $\delta(p_i, a) = S_i$; for $i = 1, 2, \dots, k$
 - Then, $\hat{\delta}(q_0, w_0) = S_1 \cup S_2 \cup \dots \cup S_k$

الشرح:

ϵ : عبارة عبارة عن Empty String

δ : تابع انتقال يأخذ Symbol وهو ϵ

$$\hat{\delta}(q, \epsilon) = q$$

حيث أن: q موجودة داخل مجموعة

إذا أخذت حالة معينة ولتكن q رمز ϵ ، فهي من الممكن أن تأخذني لأكثر من حالة، هذه الحالة q ممكن أن تعطيني S لكن ال Case هي ϵ لا تعطي S .
على الأقل خرج ال $\hat{\delta}$ هو q

إذن: هذا الكلام على الأقل يضمن أنه:

الانتقال الفارغ لحالة لم يأتها شيء (أو أتاها ϵ) تبقى بحالها

$$\hat{\delta}(q, \epsilon) = \{q\}$$

■ لنفرض جدلاً: $\hat{\delta}(q_0, w) = \{p_1, p_2, \dots, p_k\}$

حيث w عبارة عن كلمة (ولنلاحظ الفكرة هنا أن $\hat{\delta}$ ليس التقليدي بينما δ التقليدي بأخذ symbol وليس كلمة)
يتضمن أن q_0 ستخضع لمجموعة من الانتقالات، وأن q_0 بسبب w ستصل إلى حالة معينة (تذكر هذا الكلام بال NFA، حيث بال DFA: كل حالة تأخذني لحالة واحدة فقط)

Start State : q_0 أخذت w ، حيث ستعطيني different paths، والتي من الممكن أن تكون نقلتني إلى مجموعة أخرى من الحالات $p_1 \rightarrow p_k$

إذاً: $\hat{\delta}(q_0, w) \rightarrow$ مجموعة حالات

وكان من أجل كل حالة من هذه الحالات "Symbol = a"، يأخذني إلى $i = 1, 2, \dots, k$ $s_1, s_2, \dots, s_i$;
 $\hat{\delta}(q_0, w_a) = S_1 \cup S_2 \cup \dots \cup S_k$
 w is concatenated with a : تلفظ بالشكل:

الملخص للشرح:

لدينا q_0 حالة معينة، هذه الحالة نتيجة لـ word معينة دخلت لـ word معينة، وهذه الـ word المعينة لأن لدي NFA، كان من الممكن أن تسبب الانتقال إلى مجموعة حالات، فيما بعد هذه المجموعة حالات أخذت Symbol a ، عملياً النتيجة الكلية لـ q_0 تأخذ w_a ، والتي هي عبارة عن اجتماع كافة الحالات التي تأخذني لـ Symbol a للحالات الوسيطة

Language of NFA

- An NFA accepts w if there exists at least one path from the start state to an accepting (final) state that is labeled by w
- $L(N) = \{ w \mid \hat{\delta}(q_0, w) \cap F \neq \phi \}$

Language of NFA

هي عبارة عن:

NFA يقبل word إذا تواجد على الأقل مسار واحد انطلاقاً من start state عندها w تأخذني إلى final state، ولكن كيف نصل إليها؟

أي إذا كانت سلسلة من المسارات التي نصل إليها عند إدخال الـ substring w والتي هي مؤلفة من set of symbols انطلاقاً من q_0 ، سلسلة هذه الانتقالات تقاطعها مع مجموعة الحالات المقبولة النهائية هو ليس المجموعة الخالية ϕ ، وبالتالي w يتم ادراكها من قبل الـ NFA وهذه هي الـ Language of NFA

$$L(N) = \{ w \mid \hat{\delta}(q_0, w) \cap F \neq \phi \}$$

ما ندرسه الآن هو أساس الـ Modeling / النمذجة / أساس تصميم الـ Compiler / أساس الكثير من الأشياء التي سندرسها مستقبلاً

Technologies for NFAs

هل الـ NFA يتم تطبيقه على مستوى الـ Hardware بشكل مباشر من قبل؟

الجواب لا، والفكرة أنه: الـ NFA يتحول إلى DFA بعد ذلك يطبق على الـ Hardware

إن أول NFA ولهذه اللحظة وحيد يتم تطبيقه على رقائق السيليكون هو: Micron's Automata Processor، والذي هو معالج أوتومات من شركة مايكرون تم انتاجه عام 2013 ولكنه فشل فشلاً كبيراً تجارياً ولم يقوَ على تنافس شركة NVIDIA ولكنه لا يزال قيد الدراسة، والتي تنص على تطبيق الـ NFA مباشرة على السيليكون، أي بدون وجود Software Base

الهدف منها:

عمل تفرع غير حتمي، أي عملياً كل الانتقالات غير الحتمية تنفذ على المستوى التفرعي (on parallel)، وهذا يوفر الكثير من الطاقة الموجودة بالـ CPU / GPU، قابلية توسعها (Scalability) عالية جداً، حيث تدعم الاوتومات بملايين الحالات حيث أن الـ Architecture يكون فيها معقد، لكن لا نراه بالاستخدام التجاري، على مستوى التجارة فشل ولكنه لا يزال Energy

DFA vs NFA

NFA	DFA
بعض الانتقالات قد تكون غير حتمية: يمكن أن يؤدي الانتقال إلى مجموعة من الحالات	كل الحالات حتمية: كل انتقال يؤدي إلى حالة واحدة فقط
ليست كل الانتقالات الرمزية بحاجة إلى أن تعرف صراحة (إذا لم تعرف فستذهب إلى حالة خطأ، هذا فقط لتسهيل التصميم وليس له علاقة بـ "عدم الحتمية")	لكل حالة يجب تعريف انتقال لكل الرموز المدخلة الممكنة (الأبجدية) (يجب تمثيل Trap / Error / Dead State)
تقبل الإدخال إذا كانت واحدة من آخر الحالات في F	تقبل الإدخال إذا كانت آخر حالة تم الوصول إليها في المجموعة F
عادة أسهل في الإنشاء من DFA	أحياناً يكون من الصعب إنشاؤها بسبب عدد الحالات
التطبيقات العملية محدودة ولكنها في طور الظهور (مثال: معالج Micron للآلات)	التطبيق العملي ممكن
يسمح بالانتقالات ϵ	لا يسمح بالانتقالات ϵ
تتطلب حالات أقل (لكن بمسارات أكثر)	تتطلب عموماً عدد أكبر من الحالات

إن ال DFA هو حالة خاصة من ال NFA والعكس غير صحيح

Equivalence of DFA & NFA

Theorem:

A language L is accepted by a DFA if and only if it is accepted by an NFA

✍ هذا يتضمن:

كل NFA أستطيع تحويله لـ DFA و DFA هو حالة خاصة من NFA، لأن الـ NFA له سلسلة من الانتقالات، أي عملياً من أجل كلمة واحدة أستطيع رسم أكثر من مسار، قد ينتهي واحد منها إلى Accept State وقد ينتهي أكثر من واحد منها إلى Final State، عندما يكون واحد فقط يأخذنا إلى Final State عندها أستطيع أن أعبر عنه بالـ DFA

✍ تكافؤ DFA و NFA:

النظرية تقول:

تقبل اللغة L بواسطة DFA إذا وفقط إذا تم قبولها بواسطة NFA

Proof:

1- الجزء الأول (if):

نثبت ذلك عن طريق إظهار أنه يمكن تحويل كل NFA إلى DFA مكافئ

2- الجزء الثاني (only-if):

كل DFA هو حالة خاصة من NFA، حيث أن كل حالة تملك انتقالاً واحداً فقط لكل رمز من رموز الإدخال، لذلك إذا كانت اللغة تقبل بواسطة DFA فهي تقبل بواسطة NFA مكافئ

Proof for the if part:

- إذا كانت لغة L تقبل بواسطة NFA، إذا هناك DFA يقبل نفس اللغة
- إذا أعطينا أي NFA يرمز له بـ N، يمكننا بناء DFA يرمز له بـ D بحيث يكون:

$$L(N) = L(D)$$

How to convert NFA into a DFA

ملاحظة: في NFA كل انتقال يمكن أن يؤدي إلى مجموعة من الحالات
الفكرة: نمثل كل مجموعة من حالات NFA بحالة واحدة فقط في DFA

NFA to DFA by subset construction

طريقة البناء اعتماداً على المجموعات الفرعية:

لنفترض أن:

$$N = \{ Q, \Sigma, \delta_N, q_0, F_N \} \rightarrow \text{مكونات } N$$

الهدف:

بناء DFA بحيث:

$$D = \{ Q_D, \Sigma, \delta_D, \{ q_0 \}, F_D \}$$

مجموعة وليس Start State واحدة

$$L(N) = L(D)$$

خطوات البناء Construction:

1. $Q_D =$ جميع المجموعات الجزئية من Q_N (أي مجموعة القوى Power Set) $Q_D = \text{all subsets of } Q_N \text{ (i.e. power set)}$

مجموعة القوى Power Set:

مجموعة القوى لمجموعة S والتي يرمز لها بـ $P(S)$ وهي مجموعة كل المجموعات الجزئية الممكنة لـ S ، وتشمل:

- المجموعة الفارغة ϕ
- المجموعة S نفسها
- جميع التركيبات الممكنة لعناصر S

مجموعة القوى تمثل كل التراكيب الممكنة لعناصر S

- Q_D is the power set of (Q_N)

 Q_D ستكون عبارة عن set محتواة بالـ power set of (Q_N)

- Each DFA state represents a subset of NFA states

مثال:

إذا كان لدينا:

$$Q_N = \{ q_0, q_1 \}$$

فإن

$$Q_D = \{ \phi, \{ q_0 \}, \{ q_1 \}, \{ q_0, q_1 \} \}$$

عدد الحالات الموجودة بالـ power set للمجموعة S هو عدد صحيح ويساوي

$$2^{\text{الحالات عدد}} = |P(S)|$$

If S has n elements, its power set has 2^n subsets

$$\text{EX: for } S = \{ a, b \}, P(S) = \{ \phi, \{ a \}, \{ b \}, \{ a, b \} \}$$

$$\text{So } |P(S)| = 2^2 = 4$$

2. F_D (Final States of the DFA)

$F_D = \text{set of subsets } S \text{ of } Q_N \text{ s.t. } S \cap F_N \neq \phi$

F_D includes all subsets $S \subseteq Q_N$ where $S \cap F_N \neq \phi$

F_D تشمل جميع المجموعات الجزئية S المحتواة في Q_N بحث أن: $S \cap F_N \neq \phi$

A subset is final if it contains at least one final state of the NFA

تكون المجموعة جزئية نهائية إذا كانت تحتوي على الأقل على حالة نهائية واحدة من الـ NFA

مثال:

لنفترض أن:

$Q_N = \{1, 2, 3, 4\} \rightarrow$ هذه هي مجموعة الحالات الكاملة (كل الحالات الممكنة)

$F_N = \{3, 4\} \rightarrow$ (أي الحالات الناجحة أو المقبولة) NFA هذه هي الحالات النهائية في

الآن نريد أن نحدد ماهي المجموعات الجزئية S من Q_N التي نضمها داخل F_D بشرط:

أن يكون التقاطع بين S و F_N ليس فارغاً، أي:

$$S \cap F_N \neq \phi$$

لنأخذ أمثلة لمجموعات جزئية S :

1- $S = \{1, 2\} \leftarrow$ لا يوجد أي عنصر من F_N داخلها $\leftarrow S \cap F_N = \phi$ لا تدخل F_D

2- $S = \{2, 3\} \leftarrow$ يوجد العنصر 3 من: $S \cap F_N = \{3\} \leftarrow S \cap F_N \neq \phi$ تدخل F_D

3- $S = \{4\} \leftarrow$ العنصر 4 موجود في F_N تدخل F_D

4- $S = \{1, 3, 4\} \leftarrow$ تحتوي على 3 و 4 من F_N $\leftarrow S \cap F_N \neq \phi$ تدخل F_D

5- $S = \{1\} \leftarrow$ لا تحتوي على 3 أو 4 لا تدخل F_D

مما يعني: كل مجموعة تحتوي على 3 أو 4 على الأقل (في هذا المثال) تدخل ضمن F_D

انتبه: $S \subseteq Q_N$ تعني S هي مجموعة جزئية من Q_N أو أنها تحتوي بعض أو كل العناصر من Q_N

F_N تعني مجموعة فرعية مهمة داخل Q_N

3. δ_D : for each subset S of Q_N and for each input symbol a in Σ :

$$\delta_D(S, a) = \cup \delta_N(P, a)$$

شرح الرموز:

S : مجموعة الحالات في NFA (يعني مجموعة جزئية من Q_N)

a : رمز إدخال (input symbol)

$\delta_N(p, a)$: في NFA هي الحالات التي يمكن الوصول لها من الحالة p عند قراءة الرمز a

$\delta_D(S, a)$: في DFA هي الحالات التي تنتقل لها من المجموعة S عند قراءة الرمز a

المعنى البسيط:

عند تحويل NFA إلى DFA:

- بدلاً من وجود حالة واحدة، الحالة في DFA تمثل مجموعة حالات من NFA
- عندما يكون لدينا الحالة S في DFA، ونقرأ الحرف a نقوم بـ:
 - 1- تطبيق δ_N على كل حالة في S
 - 2- تجمع كل النتائج (بالاجتماع $\cup$)
 - 3- الناتج هو الحالة الجديدة في DFA

The DFA transitions to the union of all NFA states reachable from any state in S on input a

DFA ينتقل إلى اجتماع كل الحالات التي يمكن الوصول إليها في NFA من أي حالة داخل المجموعة S عند قراءة الرمز a

مثال سريع:

نفترض:

$$S = \{ q_1, q_2 \}$$

عند الرمز a :

$$\delta_N(q_1, a) = \{ q_3 \}$$

$$\delta_N(q_2, a) = \{ q_4, q_5 \}$$

إذن:

$$\delta_D(S, a) = \delta_D(\{ q_1, q_2 \}, a)$$

$$\delta_N(q_1, a) \cup \delta_N(q_2, a) = \{ q_3 \} \cup \{ q_4, q_5 \} = \{ q_3, q_4, q_5 \}$$

تمرين:

Exercise 5: NFA to DFA construction

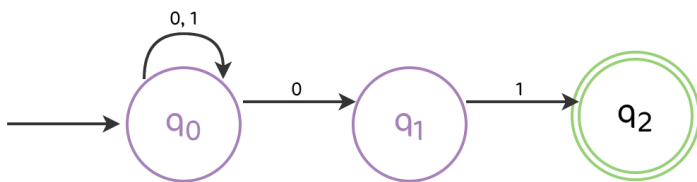
$$L = \{ w \mid w \text{ ends in } 01 \}$$

الهدف من التمرين:

تحويل NFA يعبر عن اللغة L، أي كل كلمة تنتهي بـ "01"

الخطوة 1: فهم ال NFA:

الحالات:

q₀: الابتدائيةq₁: عند قراءة "0" تنتقل إلى q₁q₂: عند قراءة "1" من q₁ تنتقل إلى q₂ (نهاية 01)

δ_N	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	ϕ	$\{q_2\}$
q_2	ϕ	ϕ

الخطوة 2: بناء ال DFA باستخدام طريقة المجموعات الجزئية (Subset Construction):

الفكرة الأساسية:

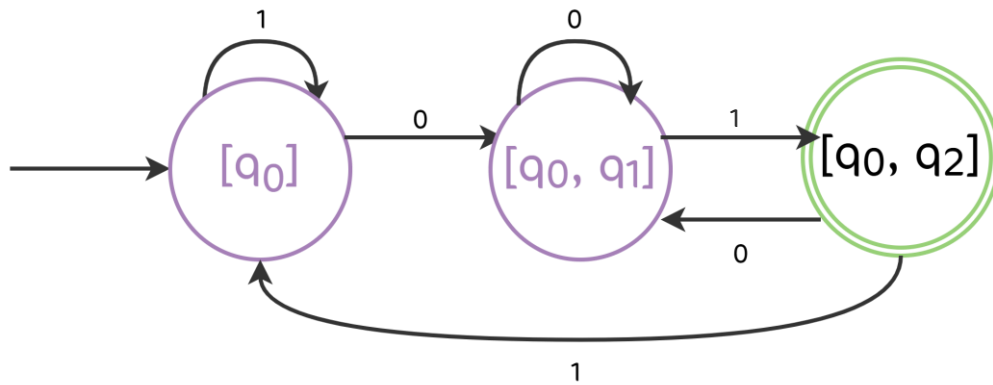
- كل حالة في DFA تمثل مجموعة من حالات NFA
- نبدأ من $\{ [q_0] \}$ لأنها البداية
- نحدد الانتقالات استناداً إلى كل مدخل (0 أو 1)
- أي مجموعة تحتوي على q_2 تعتبر نهائية (لأن q_2 حالة قبول في NFA)

الجدول الناتج للـ δ_D :

δ_D	0	1	نهائية؟
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_0]$	لا
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_2]$	لا
$[q_0, q_2]$	$[q_0, q_1]$	$[q_0]$	نعم
$[q_1]$	ϕ	$[q_2]$	لا
$[q_2]$	ϕ	ϕ	نعم
$[q_1, q_2]$	ϕ	$[q_2]$	نعم
$[q_0, q_1, q_2]$	$[q_0, q_1]$	$[q_0, q_2]$	نعم
ϕ	ϕ	ϕ	لا

كل حالة تحتوي على q_2 تعتبر حالة نهائية

حيث أن رسم DFA هو كالتالي:

الخطوة 3: نحتفظ فقط بالحالات القابلة للوصول من $[q_0]$:نبدأ من $[q_0]$ ونتبع الانتقالات:

- 1- من $[q_0]$ بـ 0 $\leftarrow [q_0, q_1]$
- 2- من $[q_0]$ بـ 1 $\leftarrow [q_0]$
- 3- من $[q_0, q_1]$ بـ 0 $\leftarrow [q_0, q_1]$
- 4- من $[q_0, q_1]$ بـ 1 $\leftarrow [q_0, q_2]$
- 5- من $[q_0, q_2]$ بـ 0 $\leftarrow [q_0, q_1]$
- 6- من $[q_0, q_2]$ بـ 1 $\leftarrow [q_0]$

إذن الحالات التي يمكن الوصول إليها من $[q_0]$: $[q_0, q_2]$ $[q_0, q_1]$ $[q_0]$

(باقي الحالات مثل $[q_1]$ ، $[q_2]$ ، $[q_1, q_2]$ لا يمكن الوصول إليها من $[q_0]$ في مثل هذا الرسم، لذلك نحذفها من DFA النهائي)

DFA النهائي المبني:

	0	1	نهائية
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_0]$	لا
$[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_2]$	لا
$[q_0, q_2]$	$[q_0, q_1]$	$[q_0]$	نعم (تحتوي على q_2)

متى تقبل DFA للكلمة؟

إذا انتهت الكلمة بـ "01" فإنها تمر بالانتقالات التي توصل $[q_2]$ ضمن الحالات المركبة وبالتالي يتم قبولها

اليكم الخطوات الثلاث والتي تعد جزء من طريقة التحويل من NFA إلى DFA باستخدام "بناء المجموعات الجزئية" (subset construction):

1. Enumerate all possible subsets

عدد كل المجموعات الجزئية الممكنة من حالات NFA

ماذا يعني؟

نفترض أن NFA فيه الحالات $Q = \{q_0, q_1, q_2\}$

المجموعات الجزئية الممكنة هي كل التراكيب الممكنة من هذه الحالات (بما فيها المجموعة الخالية):

$$S = \{ \phi, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\} \}$$

عدهم: مجموعات جزئية $2^3 = 8$

أي تتضمن هذه الخطوة توليد المجموعات الجزئية

2. Determine transitions

حدد الانتقالات بين الحالات بناء على جدول δ في NFA

كيف نقوم بها؟

لكل مجموعة جزئية S ولكل رمز إدخال a (0 أو 1):

نحسب:

$$\delta_D(S, a) = \cup \delta_N(p, a)$$

مثال:

إذا كانت $S = \{q_0, q_1\}$ و $a = 1$

- من q_0 عند 1 $\leftarrow \delta_N(q_0, 1) = \{q_0\}$

- من q_1 عند 1 $\leftarrow \delta_N(q_1, 1) = \{q_2\}$

إذن:

$$\delta_D(\{q_0, q_1\}, 1) = \{q_0, q_2\}$$

حيث:

$$\delta_D(S, a) = \delta_D(\{q_0, q_1\}, 1) =$$

$$\delta_N(q_0, 1) \cup \delta_N(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$$

أي تتضمن هذه الخطوة بناء جدول الانتقال لـ DFA

3. Return only those states reachable from $\{q_0\}$

نحتفظ فقط بالحالات التي نستطيع الوصول إليها بدءاً من $\{q_0\}$

لماذا؟

من غير الممكن الوصول فعلياً إلى كل المجموعات الجزئية من البداية، فنوفر الوقت والموارد بحذف الذي لا نستطيع الوصول إليه

كيف نقوم بذلك؟

- نبدأ من الحالة الابتدائية $[q_0]$
- نتابع كل انتقالاتها (لكل 0 و 1)
- نضيف كل حالة جديدة تنتقل إليها
- نكرر حتى تكمل كل الحالات التي يمكن الوصول إليها

مثال:

- $[q_0]$ عند 0 $\leftarrow [q_0, q_1]$
 - $[q_0, q_1]$ عند 0 $\leftarrow [q_0, q_2]$
 - $[q_0, q_2]$ عند 0 $\leftarrow [q_0, q_1]$
- فنحتفظ فقط بالحالات:

$[q_0], [q_0, q_1], [q_0, q_2]$

أي هذه الخطوة تتضمن حذف الحالات غير القابلة للوصول من $[q_0]$

ϵ NFAs

تذكر: ϵ هو العنصر الفارغ

Definition and key characteristics

- DFA does not support ϵ -transitions counter to NFA, NFA supports ϵ -transitions

أي: بالـ NFA: ϵ = Empty symbol ينقلك من State إلى State
نذكر أن:

$$\delta(q, \epsilon) = \{q\}$$

إن:

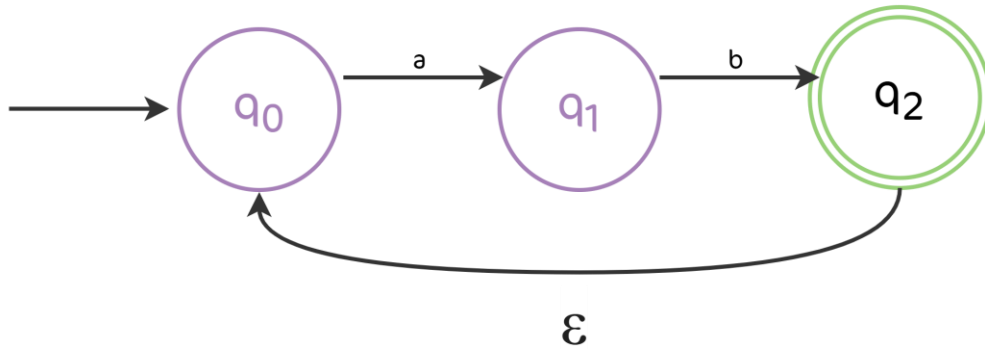
ϵ -NFA:

هي عبارة عن non-deterministic finite، تدعم الانتقالات الفارغة، بمعنى أنه نستطيع رؤية transitions معبر عنها بـ ϵ تماماً وينقلني من حالة إلى حالة، فعلياً δ من أي state عن طريق ϵ هو state نفسها وهذا ما تعلمناه في المحاضرة السابقة

ϵ - NFA حالة خاصة من NFA تحتوي ϵ - transitions وبالتالي هذا يؤدي إلى تغيير الحالة دون الحاجة لوجود input symbol يغير هذه الحالة

السؤال:

- ما هي الفائدة إذا لم تأخذ الـ state شيء (ولا رمز) أو أخذت (ϵ) empty symbol، أن تذهب لحالة أخرى؟
- ما هي الفائدة من الـ ϵ - transitions؟
- أن لا تتوقف السلسلة، وهذا يعني إضافة المزيد من المرونة في تصميم Automaton يقبل لغة معينة مثال: لدينا:



هذا يعني: ab مقبولة ✓ abab مقبولة ✓ ababab مقبولة ✓ وهكذا....

وبالتالي استطعت تصميم شيء يشبه تصميم NFA يقبل هذه اللغة بطريقة مرنة جداً
الفكرة هي أنها added flexibility، أي المرونة المضافة التي يعطيها الـ ϵ

مزايا الـ ϵ - NFA

- يبسط تصميم الأوتومات التي تتعامل مع أنماط لغوية معقدة
- لكنه لا يزيد القدرة التعبيرية عن DFA أو NFA، أي جميعهم يقبلون نفس فئة اللغات (اللغات النظامية Regular Languages)

الهدف:

إضفاء المزيد من المرونة على المرونة الموجودة بالـ NFA، لكن لا يعطي أي شيء إضافي أكثر

Formal definitions ϵ -NFA:

Defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- Q : مجموعة الحالات
- Σ : الأبجدية (دون إبسيلون)
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow p(Q)$ دالة الانتقال
- q_0 : الحالة الابتدائية
- F : الحالات النهائية

انتبه:

ϵ ابسيلون لا يكون موجود داخل الأبجدية التي يدركها $NFA - \epsilon$ وإنما بتابع الانتقال اجتماع ϵ مع Σ يعطيني الـ power set of (Q)

مفهوم الإغلاق-إبسيلون ($\epsilon - closure$)

تذكرة: متى نقول عن مجموعة معينة أنها مغلقة أما عملية معينة؟

إذا اخترت عنصرين أو أكثر لا على التعيين من المجموعة وطبقت عليهم هذه العملية، ينتج عندي عنصر ينتمي لنفس المجموعة.

على سبيل المثال:

مجموعة الأعداد الطبيعية مغلقة أمام عملية الجمع، لأن جمع عددين طبيعيين أو أكثر هو عدد طبيعي، ولكنها ليست مغلقة أمام عملية الطرح

مفهوم $\epsilon - closure$

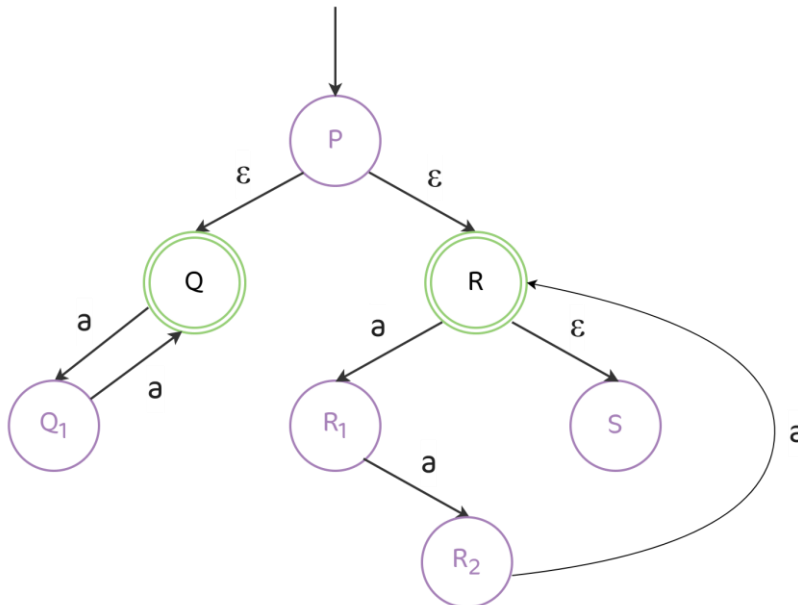
$\epsilon - closure(q)$: مجموعة تضم كل الحالات التي يمكن الوصول لها من q باستخدام 0 (تعطيني المجموعة نفسها) أو أكثر من $\epsilon - transition$

Note:

ϵ is never a member of Σ , ϵ is defined to be $(\Sigma \cup \epsilon)$

$\epsilon - closure(q) = ECLOSE(q)$

EX:



$ECLOSE(p)$: مجموعة الحالات التي يمكن الوصول إليها عند أخذ ϵ أو أكثر
لدينا الأبجدية δ هي فقط: a

$$ECLOSE(P) = \{P, Q, R, S\}$$

$$1- \delta(p, \epsilon) = P$$

$$2- P \xrightarrow{\epsilon} R \xrightarrow{\epsilon} S$$

$$3- P \xrightarrow{\epsilon} Q$$

$$ECLOSURE(R) = \{R, S\}$$

$$ECLOSURE(X) = \{X\} \quad \text{من أجل كل الحالات المذكورة}$$

For remaining 5 states (Q, Q₁, R₁, R₂, S)

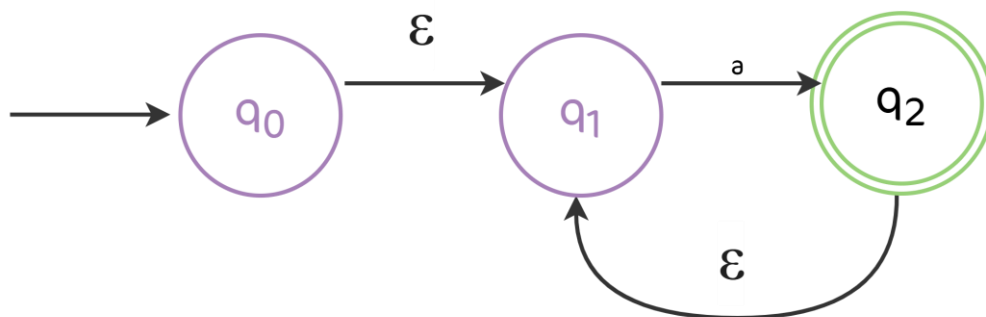
هذه الحالات لا تملك أي انتقال ϵ خارج عنها

Conversion of ϵ -NFA to equivalent DFA by implementing subset construction method

التحويل من $NFA - \epsilon$ إلى DFA عن طريق بناء المجموعات الجزئية:
الخطوات:

- 1- نحسب ϵ -closure لكل حالة
- 2- نعتبر كل مجموعة من الحالات الناتجة كـ "حالة" في الـ DFA الجديدة
- 3- ننشئ الانتقالات بين هذه الحالات الجديدة بناءً على المدخلات
- 4- نقبل هذه السلسلة إذا انتهت في مجموعة تحتوي على حالة نهائية من الـ NFA الأصلي
- هذه الطريقة يمكن أن تنتج عدداً كبيراً من الحالات

Example 1:



Original ε -NFA:

- States: $\{q_0, q_1, q_2\}$, start: q_0 , accepting state: q_2
- Transitions:

$$q_0 \xrightarrow{\varepsilon} q_1,$$

$$q_1 \xrightarrow{a} q_2,$$

$$q_2 \xrightarrow{\varepsilon} q_1,$$

 ε -closure:

سنقوم بحساب ε -closure للحالات جميعها ولكن ما سنقوم به كالتالي:

نحسب ε -closure للحالة الأولى ونطبق عليها الخطوات ثم نعود ونحسب ε -closure للثانية وهكذا.... لباقي الحالات

الحالة الأولى q_0 :

$$\varepsilon\text{-closure}(q_0) = \{q_0, q_1\}$$

q_0, q_1 : سنقوم باختبارهم على الـ input الموجود في أبجدية هذه اللغة، وفي مثالنا لدينا input واحد وهو a

$$\delta(q_0, a) = \phi$$

$$\delta(q_1, a) = q_2$$

$$\varepsilon\text{-closure}(q_2) = \{q_2, q_1\}$$

نحن بصدد أن نبني تابع الانتقال لـ NFA الجديد وليكن δ_N ، حيث أننا نحول NFA لـ NFA آخر ولكن من حالة خاصة من

(ε -NFA) إلى NFA

q_0 عند a أخذ q_2, q_1 أي:

$$\delta_N(q_0, a) = \{q_2, q_1\}$$

أي انطلاقاً من q_0 عندما نأخذ a يمكننا الذهاب لـ q_1 أو لـ q_2

تفصيل للخطوات المتبعة:

1- نحسب ε -closure للحالة

2- النتيجة التي تظهر معنا، ندرسها على الـ input بالـ Σ

3- النتيجة التي نحصل عليها نقوم بحساب ε -closure لها، ستمثل عملية أنه عندما q_0 بالـ NFA الجديد نأخذ

$a \leftarrow$ ستذهب إلى حالتين

4- نكرر هذه العملية على كافة الحالات

الحالة الثانية q_1 :

$$\varepsilon\text{-closure}(q_1) = \{q_1\}$$

$$\delta(q_1, a) = q_2$$

$$\varepsilon\text{-closure}(q_2) = \{q_2, q_1\}$$

$$\delta_N(q_1, a) = \{q_2, q_1\}$$

الحالة الثالثة : q_2

$$\varepsilon\text{-closure}(q_2) = \{q_2, q_1\}$$

$$\delta(q_2, a) = \phi$$

$$\delta(q_1, a) = q_2$$

$$\varepsilon\text{-closure}(q_2) = \{q_2, q_1\}$$

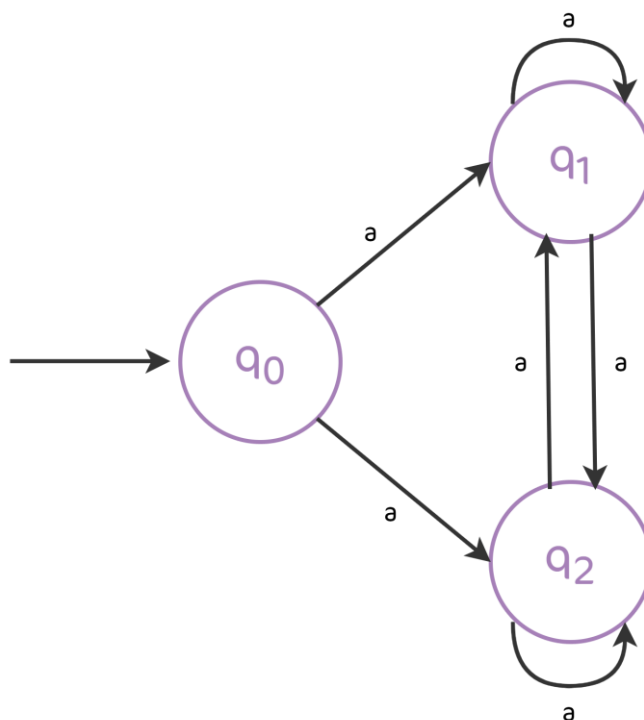
$$\delta_N(q_2, a) = \{q_2, q_1\}$$

إن الحالة الابتدائية لا DFA = $\varepsilon\text{-closure}(\varepsilon\text{-NFA})$ (الحالة الابتدائية بال NFA)

States of $\varepsilon\text{-NFA}$	$\varepsilon\text{-closure}$	input	$\varepsilon\text{-closure}$
q_0	$q_0 \xrightarrow{a} q_0$ $q_0 \xrightarrow{a} q_1$	ϕ q_2	$\{q_2, q_1\}$
q_1	$q_1 \xrightarrow{a} q_1$	q_2	$\{q_2, q_1\}$
q_2	$q_2 \xrightarrow{a} q_2$ $q_2 \xrightarrow{a} q_1$	ϕ q_2	$\{q_2, q_1\}$

ننشئ الانتقالات بين هذه الحالات الجديدة بناءً على المدخلات:

مخطط الـ NFA هو كالتالي: أي للـ δ_N



تم تحديد الحالة الابتدائية لـ DFA وهي:

$$\varepsilon\text{-closure}(q_0) = \{ q_0, q_1 \}$$

والآن لنحدد الحالة / الحالات النهائية (لنقبل السلسلة) في الـ DFA:
لنبحث عن أي مجموعة تحتوي على الحالة النهائية الأصلية في الـ NFA وهي q_2 ، فتكون الحالة النهائية:

$$\varepsilon\text{-closure}(q_0) = \{ q_0, q_1 \} \quad \times$$

$$\varepsilon\text{-closure}(q_1) = \{ q_1 \} \quad \times$$

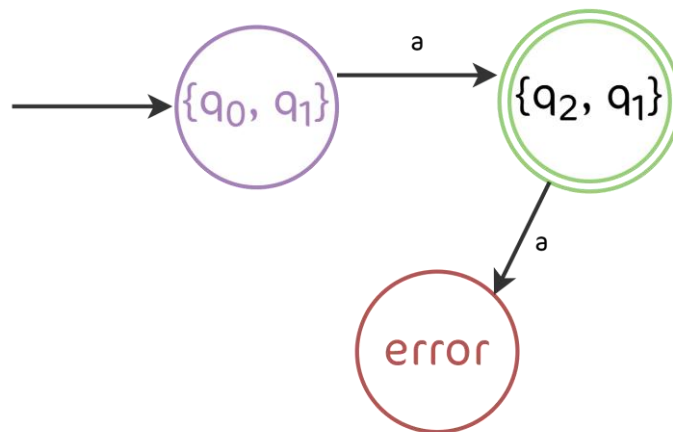
$$\varepsilon\text{-closure}(q_2) = \{ q_2, q_1 \} \quad \checkmark \text{ final state}$$

ملاحظة:

إذا كانت q_2 حالة نهائية بالـ $\varepsilon\text{-NFA}$ ، يكون $\varepsilon\text{-closure}$ هو حتماً حالة مقبولة بالـ DFA، لأن:
 $\varepsilon\text{-closure}(q_2) = \text{at least } \{ q_2 \} = \{ q_2, q_1 \}$

تم قبول هذه السلسلة لأنها انتهت في مجموعة تحتوي على حالة نهائية من NFA الأصلي

Resulting DFA:



Conversion of $\varepsilon\text{-NFA}$ to equivalent NFA by implementing elimination of $\varepsilon\text{-transitions}$ method

في هذه الفقرة سنتعلم كيف نحول آلة $\varepsilon\text{-NFA}$ التي تحتوي على انتقالات (ε) إلى آلة مكافئة بدون هذه الانتقالات وذلك بخطوات منظمة سنقوم بسردها مبدئياً ثم عن طريق المثال ستتوضح هذه الخطوات أكثر

1.

حساب ECLOSE من أجل كل حالة موجودة في الآلة الأصلية N

2.

تحديد حالات القبول الجديدة للآلة المكافئة N^* ، كيف يتم ذلك؟

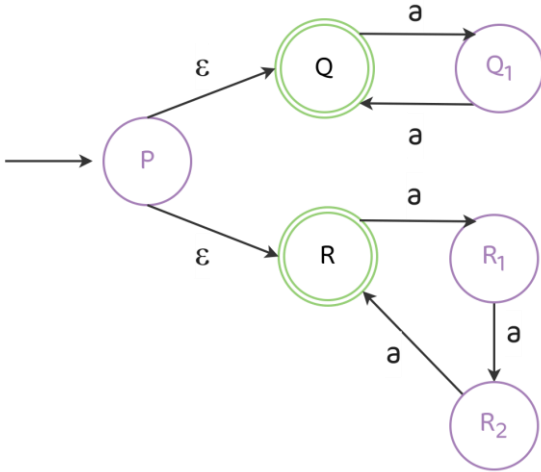
من أجل كل حالة P (أي كل حالة موجودة في الآلة الأصلية N) تكون P حالة قبول إذا كانت $Eclose(P)$ تحوي على حالة من حالات القبول الموجودة في N

3.

بناء الانتقالات الجديدة، كيف يتم ذلك؟

من أجل كل حالة P (أي كل حالة موجودة في الآلة الأصلية N) نبحث في $Eclose(P)$ إن وجدنا حالة ولنفرض أنها r لها انتقال (غير ϵ) إلى حالة q عندها نضيف انتقال من p إلى q بالرمز الذي سبب الانتقال إلى الآلة الجديدة N^*

4.

تقوم بحذف الانتقالات ϵ نهائياً ...

مثال:

قم بتحويل الآلة من نوع $NFA - \epsilon$ إلى آلة من نوع NFA باستخدام
elimination of ϵ -transitions method

2- تحديد حالات القبول الجديدة		1- حساب Eclose لكل حالة
← حالة قبول	تحتوي Q, R (حالات قبول في الآلة الأصلية)	$Eclose(P) = \{ P, Q, R \}$
← حالة قبول	تحتوي Q (حالة قبول في الآلة الأصلية)	$Eclose(Q) = \{ Q \}$
← حالة قبول	تحتوي R (حالة قبول في الآلة الأصلية)	$Eclose(R) = \{ R \}$
← ليست حالة قبول	لا تحتوي أي حالة قبول من الآلة الأصلية	$Eclose(Q_1) = \{ Q_1 \}$
← ليست حالة قبول	لا تحتوي أي حالة قبول من الآلة الأصلية	$Eclose(R_1) = \{ R_1 \}$
← ليست حالة قبول	لا تحتوي أي حالة قبول من الآلة الأصلية	$Eclose(R_2) = \{ R_2 \}$

$$\text{Eclose}(P) = \{ P, Q, R \}$$

كل انتقالاتها عبر الرمز ε

$$Q_1 \xrightarrow{a} Q$$

$$P \xrightarrow{a} Q_1$$

$$R_1 \xrightarrow{a} R$$

$$P \xrightarrow{a} R_1$$

$$\text{Eclose}(Q) = \{ Q \}:$$

من أجل Q:

$$Q \xrightarrow{a} Q_1$$

$$\text{Eclose}(R) = \{ R \}:$$

من أجل R:

$$R \xrightarrow{a} R_1$$

$$\text{Eclose}(Q_1) = \{ Q_1 \}:$$

من أجل Q_1 :

$$Q_1 \xrightarrow{a} Q$$

$$\text{Eclose}(R_1) = \{ R_1 \}:$$

من أجل R_1 :

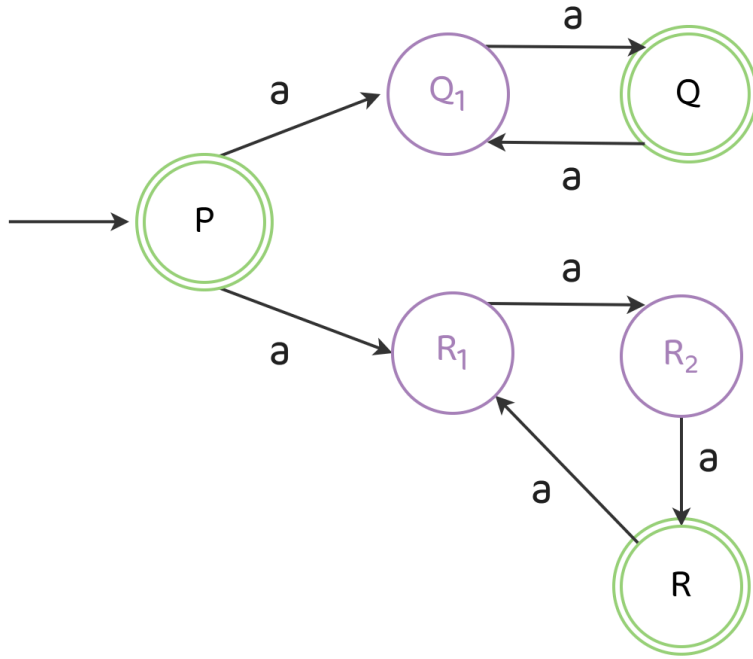
$$R_1 \xrightarrow{a} R_2$$

$$\text{Eclose}(R_2) = \{ R_2 \}:$$

من أجل R_2 :

$$R_2 \xrightarrow{a} R$$

4- نقوم بحذف انتقالات ϵ نهائياً:
فيصبح تمثيل الآلة NFA بالشكل:



Subset construction method

VS

 ϵ -Elimination method

ϵ -Elimination	Subset construction	المقارنة
التحويل لـ NFA	التحويل لـ DFA	الهدف
نفس الحالات الأصلية	حالات مركبة جديدة	التعامل مع الحالات
باستخدام ϵ -closure	بين كل المجموعات الممكنة	الانتقالات
منخفض	مرتفع جداً	خطر تضخم الحالات
يتم حذفها مباشرة	يتم دمجها في الحالات المركبة	التعامل مع ϵ
إذا كانت Eclose تحتوي واحدة	أي مجموعة تحتوي حالة نهائية	الحالات النهائية